CyberBank IDOR Vulnerability Assessment Report

Nov 11, 2026

Executive Summary

This assessment of the CyberBank web application identified four critical, authenticated-user-exploitable vulnerabilities spanning two categories: Insecure Direct Object References / Broken Object-Level Authorization (IDOR/BOLA) and broken business logic / authentication binding. All four allowed a low-privileged, authenticated customer to read or manipulate data belonging to other customers, or to defeat core financial controls, without any additional access or elevated privilege.

Severity	Count
Critical	4
High	0
Medium	0

Each finding was independently reproduced and confirmed with a unique proof-of-exploitation token (see Vulnerability Details). Left unremediated, these issues would allow any CyberBank customer to view other customers' personal and financial data, inflate their own account balance without limit, and charge purchases to other customers' payment cards.

Introduction

CyberBank is a web-based retail banking application that lets authenticated customers view account balances and transaction history, manage contacts and linked cards, transfer funds, and make card payments. This report documents a security assessment of CyberBank performed from the perspective of an authenticated, low-privileged customer attempting to access or affect other customers' accounts.

The purpose of the assessment was to evaluate CyberBank's object-level authorization and business-logic controls — specifically, whether the application correctly verifies that a user is entitled to access or act on a given resource (account, card, or transaction) before serving the request. The assessment was conducted against a dedicated, isolated test instance of CyberBank using a single authorized test account; no production systems, real customers, or real financial instruments were involved.

Methodology

The assessment followed a black-box, authenticated testing approach consistent with the OWASP Web Security Testing Guide and the OWASP API Security Top 10 methodology for access-control testing:

1. **Reconnaissance** — Logged into CyberBank with a single low-privileged test account and mapped the application's functionality (dashboard, transaction history, contacts, cards, transfers, upgrade/payment flow).

2. **Traffic analysis** — Used browser developer tools (Network and Debugger panels) to observe every API call the front end made, identify object identifiers (user IDs, account IDs, contact IDs, card IDs, transaction IDs) in requests and responses, and review the client-side JavaScript bundle to enumerate API routes not visible in normal navigation.
3. **Parameter tampering** — Captured legitimate, authenticated requests and resent them with modified object identifiers, substituting IDs belonging to other customers in place of the test account's own IDs, to test whether the server enforced ownership checks. This replay/modify step used the browser's built-in "Edit and Resend" network-request feature — functionally equivalent to a proxy tool's repeater/intercept functions.
4. **Scripted enumeration** — Where manual testing confirmed a flaw, used short JavaScript snippets run from the authenticated browser console to batch-test the vulnerability across the full set of discoverable identifiers and confirm the issue was systemic rather than a one-off.
5. **Business-logic testing** — For the funds-transfer and payment-confirmation flows, tested out-of-range and cross-entity inputs (negative amounts, mismatched transaction/authentication identifiers) that a well-behaved client would never send, to check whether validation was enforced server-side as well as client-side.
6. **Verification** — Each finding was reproduced more than once and confirmed via a distinct server-issued proof token returned only on successful exploitation, demonstrating the issue was a genuine, server-side control failure rather than a testing artifact.

All testing was performed within the scope and time window authorized for this assessment, using only the single provided test account; no credentials belonging to other customers were used or guessed.

Findings Summary

#	Finding	Category	Endpoint	Severity
1	Broken Object-Level Authorization — Customer Profile Disclosure	IDOR / BOLA	`GET /api/customer/info/{id}`	Critical
2	Broken Object-Level Authorization — Account Balance Disclosure	IDOR / BOLA	`POST /api/accounts/info`	Critical
3	Improper Input Validation — Wire Transfer Amount	Business Logic	`POST /api/accounts/transfer_to/{id}`	Critical
4	Broken Authentication Binding — 3D Secure Payment Confirmation	Broken Authentication	`POST /api/cards/confirm_payment/{id}`	Critical

All four findings were exploitable by any authenticated customer against any other customer's account, with no additional privileges, social engineering, or client-side bypass required.

Vulnerability Details

Finding 1: Broken Object-Level Authorization — Customer Profile Disclosure (Critical)

Description. The endpoint `GET /api/customer/info/{customer_id}` returns a customer's full profile — including name, linked account IDs, income, expenses, and total balance — based solely on the `customer_id` supplied in the URL. The endpoint does not verify that the requesting (authenticated) user is the owner of the profile being requested; it only checks that a valid session exists. The application's own "contacts" feature (`GET /api/customer/contacts`), intended to list a customer's transfer recipients, additionally leaks the internal `customer_id` of every contact in plaintext, giving any customer a ready-made list of valid IDs to enumerate.

Impact. Any authenticated customer can retrieve the full profile of any other customer whose ID they know or obtain via the contacts endpoint, including financial summary data that should be private. Combined with Finding 2, this enables wholesale disclosure of the bank's customer data.

Reproduction steps.

1. Authenticate as a low-privileged test customer and call `GET /api/customer/info/me` to confirm the endpoint and response shape.
2. Call `GET /api/customer/contacts` to obtain other customers' `contact_id` values (accepted by the profile endpoint as `customer_id`).
3. Request `GET /api/customer/info/{contact_id}` for a contact's ID.
4. Observe `HTTP 200` with that customer's full profile returned under the test account's own session.

Evidence. Request: `GET /api/customer/info/{victim_customer_id}`

```
{
  "message": {
    "firstname": "<redacted>",
    "lastname": "<redacted>",
    "accounts_id": ["<acct-id-1>", "<acct-id-2>"],
    "total_balance": 1500
  },
  "status": "success"
}
```

Exploitation confirmed with proof token: `1c75a844296e294a225618181ab269a9`

Finding 2: Broken Object-Level Authorization — Account Balance Disclosure (Critical)

Description. The endpoint `POST /api/accounts/info` accepts an array of internal `accounts_id` values and returns the account number, routing number, balance, and linked card IDs for each — again without verifying ownership. Account IDs for other customers are obtainable from the contacts endpoint in Finding 1, so an attacker need not guess or brute-force them.

Impact. Any authenticated customer can retrieve the account number, routing number, and current balance of any other customer's accounts. Scripted enumeration against all discovered contacts

confirmed the flaw is systemic: every reachable customer's accounts were retrievable, not an isolated case.

Reproduction steps.

1. From Finding 1, obtain a target customer's internal `accounts_id` values.
2. Submit `POST /api/accounts/info` under the test account's own session with body `{"accounts_id": ["<victim-account-id>"]}`.
3. Observe `HTTP 200` with the victim's full account record, including balance and account/routing numbers.
4. Repeat programmatically across every account ID discovered via contacts to confirm the issue is systemic.

Evidence. Request: `POST /api/accounts/info` — body `{"accounts_id": ["<victim-account-id>"]}`

```
{
  "message": [{
    "customer_id": "<victim-customer-id>",
    "number": "<account number>",
    "routing": "<routing number>",
    "balance": 1500
  }],
  "status": "success"
}
```

Enumeration across all reachable contacts returned 20 accounts belonging to 10 different customers. Exploitation confirmed with proof token: `32f0b6fa51181b0863e6f1014d1d5bb6`

Finding 3: Improper Input Validation — Wire Transfer Amount Manipulation (Critical)

Description. The wire-transfer endpoint `POST /api/accounts/transfer_to/{recipient_account_id}` accepts an `amount` field representing funds moved from the authenticated user's account to the recipient's account. The front end blocks a zero or negative amount, but this check is enforced only in the browser — the server does not independently validate that `amount` is positive.

Impact. By submitting a negative `amount` directly to the API, bypassing the UI, an authenticated customer can cause their own balance to increase with every "transfer" rather than decrease, effectively minting funds with no corresponding debit from a real source. In testing this was used to repeatedly inflate a test account's balance with no transaction limit, rate limit, or server-side sanity check observed.

Reproduction steps.

1. Identify the transfer payload shape from the application's JavaScript bundle: `POST /api/accounts/transfer_to/{account_id}` with body `{amount, raison, account_id, routing, number}`.
2. Submit a request directly to the endpoint with `"amount": -1`, using the test account's own source-account details.

3. Observe `HTTP 200` and `"status": "completed"` — the negative amount is accepted.
4. Re-query the source account's balance and confirm it increased.
5. Repeat with larger negative amounts to raise the balance past the $10,000 threshold, confirming the flaw is reliably repeatable and effectively unbounded.

Evidence. Request: `POST /api/accounts/transfer_to/{recipient_account_id}` — body `{"amount": -1, "raison": "same", "account_id": "<own-account-id>", "routing": "<own-routing>", "number": "<own-number>"}`

```
{
  "message": {"amount": -1, "status": "completed"},
  "status": "success"
}
```

Exploitation confirmed with proof token: `9b0c93f608d2aebf0588cb56cfe5e509`

Finding 4: Broken Authentication Binding — 3D Secure Payment Confirmation (Critical)

Description. CyberBank's card-payment flow initializes a transaction against a chosen card, then requires a one-time password (OTP) confirmation step intended to verify the cardholder's identity before the payment is marked confirmed: `POST /api/cards/confirm_payment/{transaction_id}` with body `{"otp", "number"}`. The server validates that the supplied `otp` matches the supplied card `number`, but does not verify that the supplied card `number` matches the card actually charged in the referenced `transaction_id`.

Impact. An authenticated customer can initiate a payment charged to another customer's card (card identifiers obtainable via Findings 1–2), then confirm that payment using their own card's valid OTP. The server accepts the mismatched confirmation and marks the transaction completed — a customer can authorize charges against a card they do not own and were never issued an OTP for, defeating the purpose of the control entirely.

Reproduction steps.

1. Retrieve another customer's card details via the object-level authorization flaw in Finding 2 (account records include linked `cards_id`).
2. Initiate a payment using the victim's card number, obtaining a `transaction_id` for a pending, unconfirmed transaction.
3. Retrieve the OTP for the test account's own card via `GET /api/cards/3dsecure/{own_card_id}`.
4. Submit `POST /api/cards/confirm_payment/{transaction_id}` (the transaction tied to the victim's card) with the test account's own OTP and own card number in the body.
5. Observe `HTTP 200` with `"status": "confirmed"` — the mismatched confirmation is accepted and the charge against the victim's card is finalized.

Evidence. Request: `POST /api/cards/confirm_payment/{transaction_id_for_victim_card_charge}` — body `{"otp": "<own-OTP>", "number": "<own-card-number>"}`

```
{
  "message": {"amount": 9.99, "status": "confirmed", "sender_payment_id":
"<victim-card-id>"},
  "status": "success"
}
```
Exploitation confirmed with proof token: `36aeff7bd9920b483a7131f602954cf7`

Additional Findings

One systemic pattern is worth calling out separately because it underlies three of the four findings: `GET /api/customer/contacts` returns the internal `customer_id`, `account_id`, and `card_id` of every contact in plaintext, with no need to guess or brute-force identifiers. This turns what might otherwise be a lower-severity IDOR (requiring ID enumeration) into a trivial one — any customer can harvest a ready-made list of valid target identifiers for the other findings directly from normal application functionality. This should be treated as a contributing root cause alongside the individual authorization failures.

Recommendations

Finding	Recommendation
1. Customer profile disclosure	Enforce server-side object-level authorization on every lookup: verify the requested `customer_id` equals the authenticated session's own ID (or that the session holds an explicit, audited permission to view it) before returning profile data; reject with `403` otherwise.
2. Account balance disclosure	Apply the same ownership check to `POST /api/accounts/info`: filter the returned accounts to only those owned by the authenticated customer, regardless of which IDs were submitted.
3. Wire transfer amount manipulation	Validate `amount` server-side as a strictly positive number within sane per-transaction and per-day limits; never trust client-side validation for financial logic. Perform balance debits/credits atomically and recompute the sender's balance from the ledger rather than trusting client-supplied deltas.
4. 3D Secure binding failure	Bind the OTP challenge to the specific `transaction_id` and card being charged at initiation time, and verify at confirmation that the supplied OTP and card number match the card recorded against that transaction — not merely that the OTP matches the card number supplied in the confirmation request.
Root cause (contacts endpoint)	Treat `customer_id`, `account_id`, and `card_id` as internal implementation details: do not return them in

Finding	Recommendation
	client-facing responses unless the endpoint also enforces ownership. Consider opaque, per-relationship reference tokens for contact-facing identifiers instead of exposing primary keys.

General recommendations

- Adopt a default-deny authorization model: every endpoint that accepts an object identifier should explicitly check ownership/entitlement before acting, rather than relying on identifiers being hard to guess.
- Add automated regression tests (ideally CI-integrated) covering every object-bearing endpoint, aligned to the OWASP API Security Top 10 BOLA category.
- Log and alert on cross-customer access attempts (a session's user ID mismatching the customer/account/card ID in the request) to detect exploitation attempts in production.
- Conduct a full authorization review of all endpoints, not only those covered by this assessment's scope, as the pattern found here (trusting client-supplied object IDs) is likely to recur elsewhere in the codebase.

Conclusion

This assessment identified four critical vulnerabilities in CyberBank, all rooted in the same underlying weakness: the application authenticates who is making a request but does not consistently verify what that user is entitled to access or do. The result is that any authenticated customer can read other customers' personal and financial data, inflate their own balance without limit, and authorize card charges they were never issued authentication credentials for.

Because these flaws require only a standard customer account and basic use of browser developer tools — no advanced exploitation skill, credential theft, or insider access — they represent an immediate and severe risk to customer data confidentiality and to the integrity of the bank's core financial controls. Remediation should prioritize server-side object-level authorization checks across every endpoint identified above, followed by a broader authorization audit of the application, before any production deployment.

References

- OWASP Top 10:2021 — A01:2021 Broken Access Control — https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP API Security Top 10:2023 — API1:2023 Broken Object Level Authorization — https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/
- CWE-639: Authorization Bypass Through User-Controlled Key — https://cwe.mitre.org/data/definitions/639.html
- CWE-840: Business Logic Errors — https://cwe.mitre.org/data/definitions/840.html
- PortSwigger Web Security Academy — Access Control Vulnerabilities — https://portswigger.net/web-security/access-control

- Tools used: browser developer tools (Network, Debugger, Console panels) for traffic inspection, request replay, and scripted enumeration.